

Logistic Regression: From Maximum Likelihood to Machine Learning

DENIS BURAKOV¹

To understand the origins of logistic regression, we must journey back to the early 1920s, when the field of statistics was, in Frank Yates’ memorable phrase, “the age of correlation and coefficients of all kinds” [20]. By 1920, statisticians had accumulated a bewildering array of techniques—least squares, method of moments, inverse probability, the χ^2 test, Pearson’s system of curves—but lacked what Yudi Pawitan calls “a firm logical foundation.” As Dermot Turing [24] observed, “The only orthodox way to measure probabilities was to take samples and observe their distribution, in the manner leading to the good old Gaussian Error Function.”

Into this confusion stepped Ronald A. Fisher² with a revolutionary idea that would transform not just statistics, but eventually machine learning, artificial intelligence, and the vast majority of probabilistic learning algorithms. His method of maximum likelihood, crystallized in the landmark 1922 paper “On the Mathematical Foundations of Theoretical Statistics” [11], provided the missing foundation. What Fisher could not have foreseen was that this same principle would, decades later, power everything from credit card approvals and spam filters to medical diagnoses and large language models like Claude and ChatGPT—all trained via maximum likelihood to estimate probabilities.

The path from Fisher’s theoretical breakthrough to today’s ubiquitous logistic regression winds through three distinct phases: the establishment of maximum likelihood as the gold standard for parameter estimation, the development of probit analysis for dose-response studies, and finally Joseph Berkson’s elegant simplification that gave us the logistic function. Each phase built upon the previous, creating a mathematical framework that bridges classical statistics and contemporary machine learning.

¹I am grateful to Andrija Djurovic for his thoughtful comments on an earlier draft of this chapter.

²**Disclaimer:** Ronald A. Fisher was a known proponent of eugenics, a field often pervaded by prejudice against racial, ethnic, and disabled groups. The online publication of this material for scholarly research purposes is not an endorsement of those views nor a promotion of eugenics in any way.

This chapter traces that remarkable journey, showing how Fisher's fundamental insight about likelihood evolved into one of the most widely used algorithms in data science. We'll see how the same mathematical principles that Fisher used to analyze simple binomial problems now govern gradient boosting machines and neural networks. Understanding this evolution reveals why logistic regression remains not just relevant but essential—it's the mathematical DNA that runs through the entire family of modern learning algorithms.

What should the reader expect from this chapter?

This chapter is organized around two broad themes: *estimation* and *uncertainty estimation*. In the first part, we trace the history of maximum likelihood and logistic regression, then explore the mechanics of estimation—from the closed-form logistic model to its connection with the Weight of Evidence (WOE) framework introduced in the previous chapter. We extend these ideas to multinomial logistic regression, showing both a closed-form solution and its WOE equivalence.

The second part focuses on quantifying uncertainty. We begin with the classical *delta method* for constructing confidence intervals and proceed to a modern approach—*conformal prediction*—using Pearson residuals as conformity scores.

The chapter concludes with historical notes, reflecting on how these ideas bridge traditional statistics and machine learning.

2.1 Maximum Likelihood: Fisher's Revolutionary Framework

The Statistical Landscape of the 1920s

In the 1920s, the dominant approach in the emerging field of statistics was what we now call “inverse probability”—essentially Bayes' theorem applied to parameter estimation. This method had fallen into disrepute due to philosophical objections about prior probabilities.

Into this theoretical void came Fisher's insight, which he had been developing since his early 1912 paper *On an Absolute Criterion for Fitting Frequency Curves* [9]: instead of asking “What is the probability that the parameter equals θ given the data?” (the inverse probability approach), we should ask “What is the probability of observing this data given that the parameter equals θ ?” (the likelihood approach). This seemingly subtle shift revolutionized statistical inference.

Fisher's influence would prove so profound that, as I.J. Good [13] later noted, “In 1950 the Bayesian approach was unpopular, largely because of Fisher's influence.” Even Harold Jeffreys, one of the founders of modern Bayesian statistics, acknowledged Fisher's contribution in his influential *Theory of Probability* [15], describing likelihood as “a convenient term introduced by Professor R. A. Fisher.”



Figure 2.1: Ronald A. Fisher (1890-1962), whose 1922 paper “On the Mathematical Foundations of Theoretical Statistics” established maximum likelihood estimation as the cornerstone of modern statistical inference—unknowingly laying the mathematical foundation for artificial intelligence.

Fisher’s 1922 Breakthrough: The Binomial Example

Much like Alan Turing transformed probabilistic inference into manageable arithmetic by working with log-odds, Fisher revolutionized parameter estimation by working with the log-likelihood. Taking logarithms converted products of probabilities into sums, making both the calculations tractable and the optimization problem clearer.

In his 1922 paper [11], Fisher illustrated maximum likelihood with the simplest possible case—one that would later become the foundation of logistic regression. He wrote:

“This question naturally first arose, not with respect to populations distributed in frequency curves and surfaces, but with respect to a population regarded as divided into two classes only, in fact in problems of probability. A certain proportion, p , of an infinite population is supposed to be of a certain kind, e.g., ‘successes,’ the remainder are then ‘failures.’ A sample of n is taken and found to contain x successes and y failures.”

The chance of obtaining such a sample is:

$$L(p) = \frac{n!}{x! y!} p^x (1 - p)^y \quad (2.1)$$

Fisher then applied his method of maximum likelihood³:

$$S(\log f) = x \log \hat{p} + y \log(1 - \hat{p}) \quad (2.2)$$

Differentiating with respect to p and setting equal to zero, so:

$$\frac{x}{p} = \frac{y}{1 - \hat{p}}, \quad \text{or} \quad \hat{p} = \frac{x}{n}. \quad (2.3)$$

This yields the maximum likelihood estimate:

$$\hat{p} = \frac{x}{n} \quad (2.4)$$

In essence, Fisher showed that the best estimate of a probability is simply the observed proportion—an intuitive result that emerges naturally from his likelihood framework. This simple example would become the building block for all of logistic regression.

Figure 2.2 illustrates this with a concrete example where the true parameter value is $p = 0.3$.

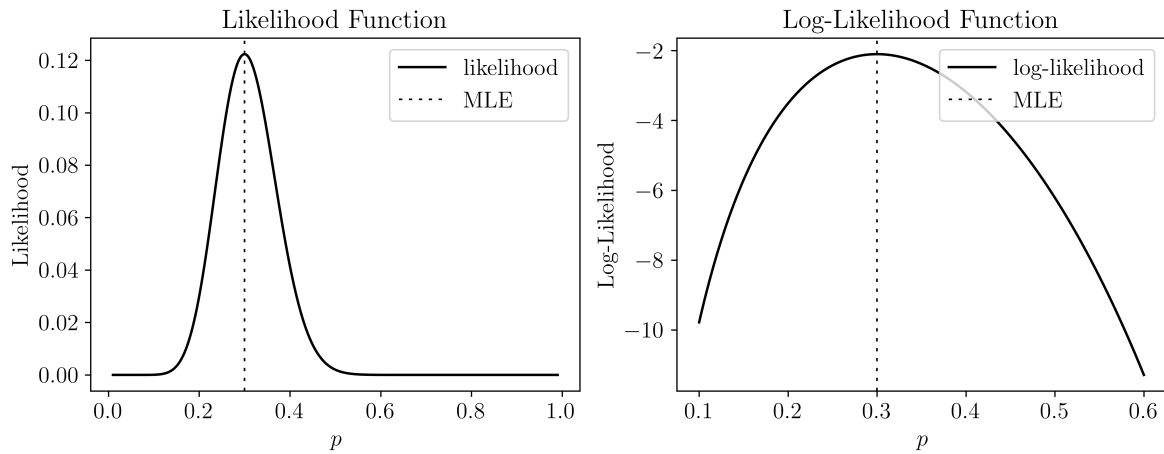


Figure 2.2: Likelihood and log-likelihood functions for a binomial parameter p . The left panel shows the likelihood function $L(p)$, while the right panel shows the log-likelihood function $\log L(p)$. Both functions peak at the true parameter value $p = 0.3$, demonstrating how maximum likelihood estimation recovers the correct parameter. The log-likelihood transformation (right) creates a more convenient optimization landscape while preserving the location of the maximum.

2.2 Why Logits Over Probits?

While Fisher was developing maximum likelihood theory, applied researchers faced a series of pressing practical problems. In the 1930s, the agricultural and pharmaceutical industries desperately needed to understand dose-response relationships: How much insecticide kills pests? What drug dosage is effective? How do we measure toxicity?

The challenge was that simple linear relationships rarely worked. Researchers observed S-shaped curves where low doses had little effect, middle doses showed rapid increases in response,

³Using S for summation, currently denoted $\{\Sigma\}$, was a common notation at the time of writing. We will refer to this expression as log-likelihood function.

and high doses approached 100% response. The question was: how do you fit a mathematical curve to this pattern?

Chester Ittner Bliss, working under Fisher's guidance at the Galton Laboratory, tackled this problem head-on. As Bliss himself wrote [2], the goal was clear:

“The fitting of a dosage-mortality curve to a series of experimental observations, however crude or refined the technique, is an attempt to infer, from a limited number of individuals, the ‘true’ empirical relationship of dosage and mortality for a given toxic agent in an infinitely larger population from which they represent only a sample.”

Bliss's breakthrough was to recognize that if individual tolerance levels followed a normal distribution, then the cumulative response would follow the cumulative normal curve. He called his transformed units “probits” (probability units) and showed that:

$$\text{probit}(p) = \Phi^{-1}(p) = \alpha + \beta \log(\text{dose}) \quad (2.5)$$

This transformation converted the S-shaped dose-response curve into a straight line, making it easy to fit using standard regression techniques. As Bliss [3] noted, this approach offered “greater ease in determining the straight line of best fit by the simple regression equation.”

The method was revolutionary for the time because it solved the curve-fitting problem that had plagued researchers for decades. By 1940, probit analysis had become the standard method for analyzing binary response data in biology, agriculture, and medicine.

But there was a problem: the calculations were extraordinarily tedious. Fitting probit models required looking up values in normal distribution tables, iterative calculations by hand, and complex adjustments for edge cases. As D.J. Finney [7] noted, Fisher's adjustments for zero response rates in his appendix to Bliss's paper [10] “must have mystified many mathematicophobic biologists!”

The computational burden was so severe that the probit method, introduced by C.I. Bliss in the 1930s, originally defined probits with 5 added to avoid negative values when using lookup tables. D.J. Finney's influential 1947 text *Probit Analysis* [8] adopted this convention, cementing it as the standard approach. As Collett [5] later explained: “The original definition of a probit [with 5 added] was primarily to avoid having to work with negative probits... This definition is still used in some quarters, but in the major statistical software packages for what is referred to as probit analysis, probits are defined without the addition of 5.” This seemingly minor adjustment reveals just how cumbersome hand calculations had become.

Into this computational challenge stepped Joseph Berkson with a simple but radical proposal: what if we used a different S-shaped curve that was mathematically simpler?

In his 1944 paper “Application of the Logistic Function to Bio-assay,” [1] Berkson proposed adapting the logistic function for statistical analysis:

$$P(\text{response}) = \frac{1}{1 + e^{-(\alpha + \beta x)}} \quad (2.6)$$

Berkson wasn't inventing the logistic function from scratch—in fact, it had been known since the 19th century, where it was used to model population growth and autocatalytic chemical

reactions [6]. But Berkson was the first to recognize its potential as a simpler alternative to the normal distribution for dose-response analysis.

Berkson's motivation was refreshingly practical. As he wrote [1]:

“I believe that the work of fitting the logistic as given here is considerably simpler than that of fitting the normal curve by probits and maximum likelihood as advocated by Bliss and Fisher.”

Berkson identified three key advantages of the logistic function that would ultimately make it dominant:

- **Computational Simplicity:** The logistic function has a simple closed-form derivative, eliminating the need for complex normal distribution tables and iterative approximations.
- **Equivalent Performance:** When Berkson tested both methods on real data, he found that “either the results are practically the same, or there is an advantage in favor of the logistic.”
- **Mathematical Tractability:** The logit transformation $\log\left(\frac{p}{1-p}\right)$ produces log-odds that exponentiate to interpretable odds ratios, unlike probit parameters which represent changes in standard normal deviates.

What Berkson could not have foreseen was that his “simpler” alternative would become the foundation of the computer age. When digital computers emerged in the 1960s and entered such areas as natural language processing in the late 1970s [21], the computational advantages of the logistic function made it the natural choice for probabilistic modeling.

From Parameter Estimation to Loss Functions

What Fisher discovered was more profound than just a new parameter estimation method—he had identified the fundamental principle underlying all learning from data.

From Fisher's Binomial Likelihood to Modern Machine Learning

Fisher's 1922 binomial example [11] established the maximum likelihood principle. We can now see that this is mathematically equivalent to minimizing negative log-likelihood. For his simple case:

$$\text{Maximize: } x \log p + y \log(1 - p) \quad (2.7)$$

is the same as:

$$\text{Minimize: } -x \log p - y \log(1 - p) \quad (2.8)$$

When we extend this to individual observations with $y_i \in \{0, 1\}$ and predicted probabilities $\hat{p}_i$, Fisher's principle becomes:

$$\text{Minimize: } -\sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (2.9)$$

This is precisely the **log loss function** (cross-entropy loss) used throughout modern machine learning:

$$\text{Log Loss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (2.10)$$

The only difference is the averaging factor $\frac{1}{n}$, which doesn't affect optimization. **This means that** every time a neural network or gradient boosting model is trained for classification, it's using Fisher's 1922 principle—maximum likelihood estimation in disguise. The connection between classical statistics and modern AI runs deeper than most practitioners realize.

Log Loss in Modern Machine Learning

Log loss penalizes confident wrong predictions exponentially more than uncertain ones. For example, a model that predicts a probability of 0.19 for the positive class ($Y = 1$) receives a loss of 1.66, while predicting 0.79 yields a much smaller log loss of 0.23. The same principle applies symmetrically to the negative class ($Y = 0$); here, we focus on $Y = 1$ for illustration.

Actual class	Predicted probability for class 1	Active term	Log loss
1	0.19	$-\log(0.19)$	1.66
1	0.79	$-\log(0.79)$	0.23

Table 2.1: Log Loss Examples: The Effect of Penalizing Wrong Predictions

We can also represent Log Loss visually in the following diagram:

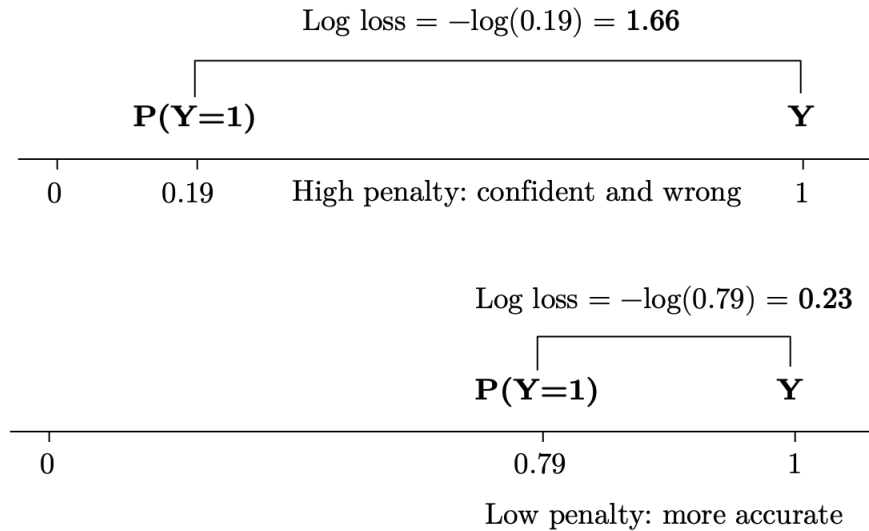


Figure 2.3: Log Loss Function Properties

Predictions that assign higher probability $P(Y = 1)$ to the actual observed outcomes of class 1 ($Y = 1$), incur smaller log loss values, whereas less accurate predictions are penalized more heavily. As a proper scoring rule, log loss reaches its minimum when predicted probabilities reflect the true likelihood of events.

A Fundamental Property: Why MLE Works

At the maximum likelihood estimate, Fisher's framework ensures a remarkable balance condition:

A Key Property of MLE

At the maximum likelihood estimate (MLE),

$$\sum_{i=1}^n y_i = \sum_{i=1}^n \hat{p}_i.$$

This means the total observed events exactly equals the total predicted events—the model's expected frequency matches the empirical frequency. **Intuitively**, this says that a well-fitted model should predict, on average, the same number of events as actually occurred⁴. **In practice**, this balance condition serves as a sanity check: if your model predicts 100 defaults but you observed 150, something is off about the fit.

To understand why this property holds, consider a concrete example with 12 trials and 3 successes. At the maximum likelihood estimate $\hat{p} = 3/12 = 0.25$, we can examine both the binomial and Bernoulli formulations:

Binomial Likelihood:

Parameter	Value
Successes (k)	3
Trials (n)	12
MLE estimate ($\hat{p}$)	0.25
Binomial coefficient $\binom{12}{3}$	220
Log binomial coefficient	5.394
Binomial log-likelihood	−1.354
Bernoulli log-likelihood	−6.748

Bernoulli Likelihood (Individual Trials):

Trial	Outcome (y_i)	Predicted ($\hat{p}_i$)	Log Likelihood
1	1	0.25	−1.386
2	1	0.25	−1.386
3	1	0.25	−1.386
4–12	0	0.25	−0.288
Sum	$\sum y_i = 3$	$\sum \hat{p}_i = 3.00$	−6.748

The key observation: $\sum_{i=1}^{12} y_i = 3 = \sum_{i=1}^{12} \hat{p}_i = 12 \times 0.25 = 3.00$

⁴When fitting logistic regression models, this property will only hold when we fit the model with an intercept term.

The difference between the binomial log-likelihood (-1.354) and Bernoulli log-likelihood (-6.748) is exactly $\log \binom{12}{3} = \log(220) = 5.394$. This binomial coefficient term is constant with respect to the parameter p being estimated, so it cancels out when taking derivatives to find the maximum—both formulations yield the same MLE. **This means that** whether we think of our data as 12 individual coin flips or as a single binomial observation, maximum likelihood gives us the same answer. This equivalence will prove crucial when we move to logistic regression.

2.3 Solving Logistic Regression By Hand

In this section, we demonstrate logistic regression estimation with maximum likelihood through a concrete example that provides intuitive understanding of the method.

Consider a small credit portfolio where we want to predict default based on a single risk factor: whether a customer has been delinquent (missed a payment) for more than five days in the past year.

Risk Factor	Default ($y = 1$)	No Default ($y = 0$)	Total
Delinquent > 5 days ($x = 1$)	80	100	180
Delinquent ≤ 5 days ($x = 0$)	20	500	520
Total	100	600	700

Table 2.2: Credit Risk Dataset: Delinquency Status and Default Outcomes

This gives us a classic 2×2 contingency table that we can analyze using logistic regression. Our model will be:

$$\log \left(\frac{P(\text{default}|\text{delinquent})}{1 - P(\text{default}|\text{delinquent})} \right) = \beta_0 + \beta_1 x \quad (2.11)$$

where $x = 1$ if delinquent > 5 days, and $x = 0$ otherwise.

This data format represents a common approach in logistic regression: one-hot encoding of categorical variables. One category serves as the reference (captured by the intercept β_0), while the coefficient β_1 estimates the effect of moving from the reference category to the other.

This example is a special case where a maximum likelihood estimate is a simple function of the data, known as a “closed-form solution.” [4]. Gelman et al. [12] discuss this example in the context of the normal approximation to the likelihood for logistic regression parameters.

Problem Setup

For our 2×2 table, we have two groups with different default probabilities:

- **Group 0 (low delinquency):** $p_0 = P(\text{default}|x = 0)$
- **Group 1 (high delinquency):** $p_1 = P(\text{default}|x = 1)$

Maximum Likelihood Estimate

The parameter of primary interest is the log-odds ratio, θ , which is the MLE:

$$\theta = \log \left(\frac{p_1/(1-p_1)}{p_0/(1-p_0)} \right) = \log \left(\frac{p_1}{1-p_1} \right) - \log \left(\frac{p_0}{1-p_0} \right) \quad (2.12)$$

Empirical Logits and Standard Errors

Using empirical logit approach, we estimate θ by:

$$\hat{\theta} = \log \left(\frac{y_1}{n_1 - y_1} \right) - \log \left(\frac{y_0}{n_0 - y_0} \right) \quad (2.13)$$

The standard error of this estimate is:

$$SE(\hat{\theta}) = \sqrt{\frac{1}{y_1} + \frac{1}{n_1 - y_1} + \frac{1}{y_0} + \frac{1}{n_0 - y_0}} \quad (2.14)$$

Calculations

Let's work through the calculations step by step using our dataset.

Step 1: Calculate Group-Specific Probabilities

For the high delinquency group ($x = 1$):

$$\hat{p}_1 = \frac{80}{180} = 0.4444 \quad (2.15)$$

$$\text{odds}_1 = \frac{0.4444}{1 - 0.4444} = \frac{0.4444}{0.5556} = 0.8000 \quad (2.16)$$

$$\log\text{-odds}_1 = \log(0.8000) = -0.2231 \quad (2.17)$$

For the low delinquency group ($x = 0$):

$$\hat{p}_0 = \frac{20}{520} = 0.0385 \quad (2.18)$$

$$\text{odds}_0 = \frac{0.0385}{1 - 0.0385} = \frac{0.0385}{0.9615} = 0.0400 \quad (2.19)$$

$$\log\text{-odds}_0 = \log(0.0400) = -3.2189 \quad (2.20)$$

Step 2: Calculate Logistic Regression Coefficients

Using the closed-form solution for binary logistic regression:

$$\hat{\beta}_0 = \log\text{-odds}_0 = -3.2189 \quad (2.21)$$

$$\hat{\beta}_1 = \log\text{-odds}_1 - \log\text{-odds}_0 = -0.2231 - (-3.2189) = 2.9957 \quad (2.22)$$

Step 3: Calculate Standard Errors

For the intercept (β_0):

$$SE(\hat{\beta}_0) = \sqrt{\frac{1}{y_0} + \frac{1}{n_0 - y_0}} = \sqrt{\frac{1}{20} + \frac{1}{500}} = \sqrt{0.0520} = 0.2280 \quad (2.23)$$

For the slope (β_1):

$$SE(\hat{\beta}_1) = \sqrt{\frac{1}{y_1} + \frac{1}{n_1 - y_1} + \frac{1}{y_0} + \frac{1}{n_0 - y_0}} \quad (2.24)$$

$$= \sqrt{\frac{1}{80} + \frac{1}{100} + \frac{1}{20} + \frac{1}{500}} \quad (2.25)$$

$$= \sqrt{0.0745} = 0.2729 \quad (2.26)$$

Step 3: Calculate Standard Errors

For the intercept (β_0):

$$SE(\hat{\beta}_0) = \sqrt{\frac{1}{y_0} + \frac{1}{n_0 - y_0}} = \sqrt{\frac{1}{20} + \frac{1}{500}} = \sqrt{0.0520} = 0.2280 \quad (2.27)$$

For the slope (β_1):

$$SE(\hat{\beta}_1) = \sqrt{\frac{1}{y_1} + \frac{1}{n_1 - y_1} + \frac{1}{y_0} + \frac{1}{n_0 - y_0}} \quad (2.28)$$

$$= \sqrt{\frac{1}{80} + \frac{1}{100} + \frac{1}{20} + \frac{1}{500}} \quad (2.29)$$

$$= \sqrt{0.0745} = 0.2729 \quad (2.30)$$

When estimating the MLE for logistic regression parameters, the standard error needs to account for the variance in all parts of the contingency table. For β_0 (the log-odds when $x = 0$), we only need data from the low delinquency ($x = 0$) group (defaults, y_0 , and non-defaults, $n_0 - y_0$). For β_1 (the log-odds ratio), we need data from both groups, hence all four cells contribute to its variance.

Step 4: Calculate Test Statistics and P-values

Wald test statistics:⁵

$$z_0 = \frac{\hat{\beta}_0}{SE(\hat{\beta}_0)} = \frac{-3.2189}{0.2280} = -14.12 \quad (2.31)$$

$$z_1 = \frac{\hat{\beta}_1}{SE(\hat{\beta}_1)} = \frac{2.9957}{0.2729} = 10.98 \quad (2.32)$$

P-values (two-tailed):

$$p_0 = 2 \times \Phi(-|z_0|) = 2 \times \Phi(-14.12) < 0.0001 \quad (2.33)$$

$$p_1 = 2 \times \Phi(-|z_1|) = 2 \times \Phi(-10.98) < 0.0001 \quad (2.34)$$

Step 5: Construct Wald Confidence Intervals

Wald 95% confidence intervals (CI) using $z_{0.025} = 1.96$ are:

⁵Under the null hypothesis $H_0: \beta = 0$, the Wald statistic is asymptotically standard normal. The two-tailed p-value is computed as $p = 2 \Phi(-|z|)$, where Φ denotes the cumulative distribution function of the standard normal distribution.

$$\triangleright \beta_0 : \text{CI}_0 = -3.2189 \pm 1.96 \times 0.2280 = [-3.666, -2.772]$$

$$\triangleright \beta_1 : \text{CI}_1 = 2.9957 \pm 1.96 \times 0.2729 = [2.461, 3.531]$$

Having calculated each component step-by-step, we can now assemble the complete report that statistical packages report automatically:

Parameter	Estimate	Std. Error	Wald Statistic	P-value	Lower CI	Upper CI
Intercept (β_0)	-3.219	0.228	-14.12	< 0.001	-3.666	-2.772
Delinquent (β_1)	2.996	0.273	10.98	< 0.001	2.461	3.531

- **Intercept ($\beta_0 = -3.219$):** The log-odds of default for customers with low delinquency (≤ 5 days). This corresponds to a probability of $\frac{e^{-3.219}}{1+e^{-3.219}} = 0.0385$ or 3.85%.
- **Slope ($\beta_1 = 2.996$):** The increase in log-odds when moving from low to high delinquency. The odds ratio is $e^{2.996} = 20.0$, meaning customers with high delinquency have 20 times higher odds of default.

Weight of Evidence (WOE) as Centered Logistic Regression

Unlike standard logistic regression with one-hot encoded categories, which measures effects relative to a reference category, WOE recenters the model around the overall sample log-odds⁶. Instead of asking “how much higher/lower than the reference group?”, WOE asks “how much higher/lower than the population average default rate?”.

We will analyze the same dataset using Weight of Evidence (WOE) values to show the mathematical equivalence. Using our dataset (80 defaults out of 180 high delinquency, 20 defaults out of 520 low delinquency), we can calculate the needed components to prove the equivalence of differences in WOE values to the logistic regression coefficient, $\hat{\beta}_1$:

Link Between WOE and Logistic Regression: $\Delta\text{WOE} = \hat{\beta}_1$

Overall log-odds (default rate):

$$p = \frac{100}{700} = 0.1429, \quad \text{logit}(p) = \log\left(\frac{0.1429}{1 - 0.1429}\right) = -1.7918$$

WOE in favor of default:

$$\text{High delinquency group } (x = 1) : \text{WOE}_1 = -0.2231 - (-1.7918) = 1.5686$$

$$\text{Low delinquency group } (x = 0) : \text{WOE}_0 = -3.2189 - (-1.7918) = -1.4271$$

Relationship to logistic regression coefficient:

$$\Delta\text{WOE} = \text{WOE}_{x=1} - \text{WOE}_{x=0} = 1.5686 - (-1.4271) = 2.9957 = \hat{\beta}_1$$

⁶For a single categorical predictor (the univariate case), this is simply a different way to express the same linear relationships. However, with *multiple categorical features*, standard logistic regression's coefficients depend on the choice of reference categories, while WOE coefficients remain consistently centered around the overall event rate.

This matches our maximum likelihood estimate, demonstrating the mathematical equivalence between WOE and logistic regression. Figure 2.4 provides a visual illustration of this.

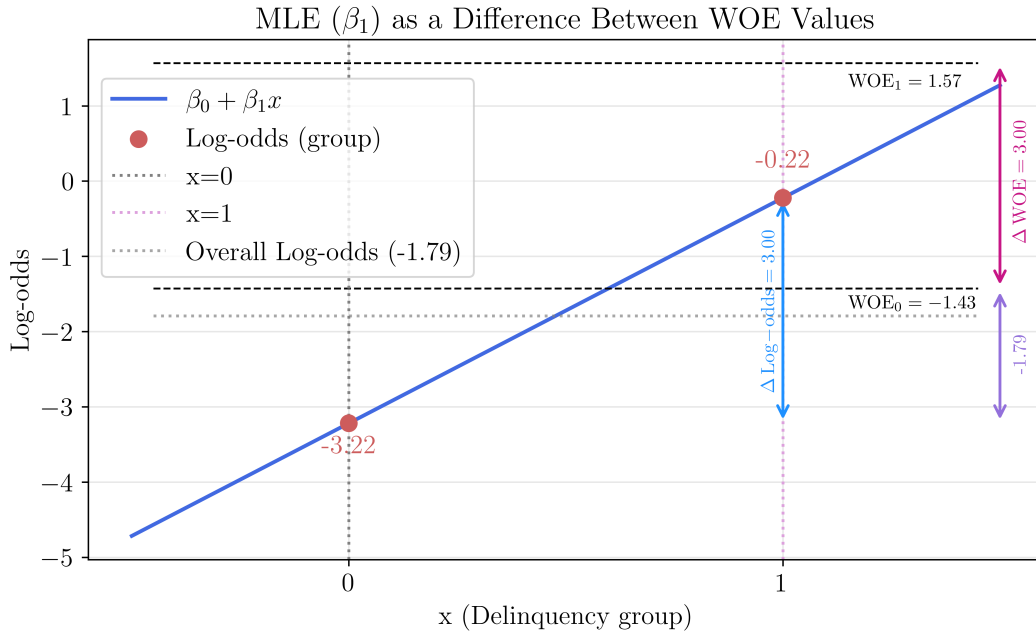


Figure 2.4: MLE as a Difference between Overall Log-Odds and WOE

WOE Standard Errors

Now that we've calculated our logistic regression results, let's understand the standard errors in the context of WOE.

The standard error of WOE for a category equals the standard error of that category's log-odds, because subtracting a constant (the overall log-odds) does not affect variance:

$$\text{Var}(\text{WOE}_i) = \text{Var}(\text{log-odds}_i - \text{overall log-odds}) = \text{Var}(\text{log-odds}_i) \quad (2.35)$$

Therefore: $\text{SE}(\text{WOE}_i) = \text{SE}(\text{log-odds}_i)$ for each category i .

Two Methods for Calculating WOE Variance:

Method	Formula
Method 1: Direct from counts	$\text{Var}(\text{WOE}_j) = \frac{1}{n_{j,\text{events}}} + \frac{1}{n_{j,\text{non-events}}}$
Method 2: From bin size and event rate	$\text{Var}(\text{WOE}_j) = \frac{1}{n_j \times p_j \times (1-p_j)}$

Both methods yield identical results. For our dataset:

- **High delinquency group:** $\text{SE}(\text{WOE}_1) = \sqrt{\frac{1}{80} + \frac{1}{100}} = 0.1500$
- **Low delinquency group:** $\text{SE}(\text{WOE}_0) = \sqrt{\frac{1}{20} + \frac{1}{500}} = 0.2280$

The logistic regression standard errors we calculated earlier can now be understood through their WOE equivalents:

Intercept Standard Error:

$$SE(\hat{\beta}_0) = 0.2280 = SE(WOE_0) \quad (2.36)$$

Slope Standard Error:

$$SE(\hat{\beta}_1) = \sqrt{SE(WOE_0)^2 + SE(WOE_1)^2} \quad (2.37)$$

$$= \sqrt{0.2280^2 + 0.1500^2} \quad (2.38)$$

$$= \sqrt{0.0520 + 0.0225} \quad (2.39)$$

$$= \sqrt{0.0745} = 0.2729 \quad (2.40)$$

This demonstrates the mathematical equivalence between the two approaches.

Figure 2.5 provides a visual illustration of the connection between standard errors of the log odds and the β_1 's standard error.

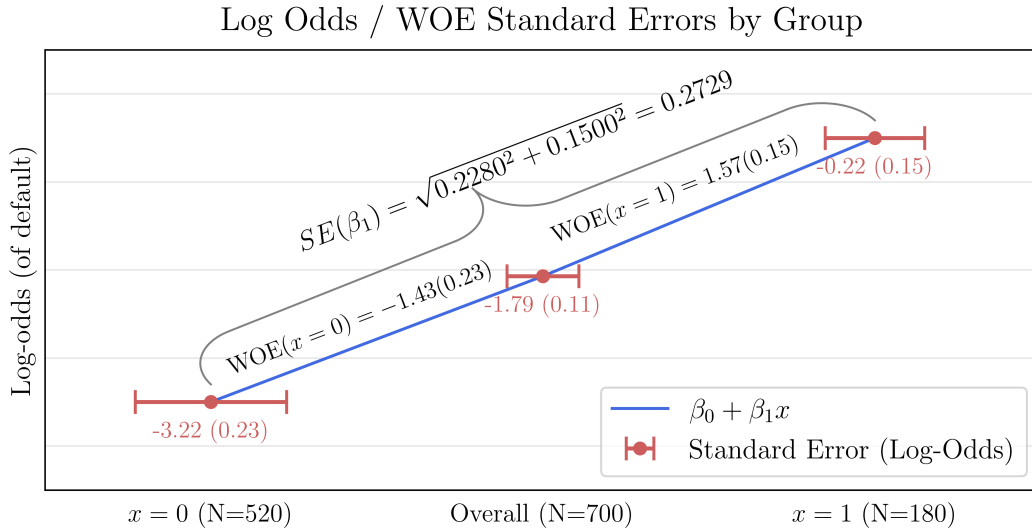


Figure 2.5: Visualization of the relationship between group-level log-odds standard errors (red bars) and the resulting slope standard error of the logistic regression model (curly brace).

2.4 Solving Logistic Regression with Iterative Optimization

The previous section demonstrated a remarkable property: when using Weight of Evidence (WOE) transformations with discretized features, we can obtain closed-form solutions for maximum likelihood estimates. This analytical tractability is one of WOE's key advantages in credit scoring applications.

However, this convenience is the exception rather than the rule. In real-world problems with multiple predictors, continuous variables, or complex feature interactions, closed-form solutions vanish. We must instead rely on iterative optimization algorithms—the computational foundation underlying modern logistic regression implementations. Having established both the analytical and computational approaches to logistic regression, we now turn to a practical application that leverages these techniques: **credit score calibration**.

The Gradient Descent Approach

The logistic function’s non-linearity in its parameters means that, except for special cases such as binary predictors with grouped data, we cannot solve for maximum likelihood estimates in closed form. Instead, logistic regression relies on iterative optimization methods that follow the gradient of the log-likelihood to find the parameter values that maximize it.

To see how this works in practice, consider a simulated dataset with 200 loan applications where we calibrate the internal bank probability of default (PD) as a function of a credit bureau score expressed in log-odds form—that is, the log of the external PD. This setup reflects a common scenario in which internal risk drivers are unavailable or limited, so an external PD estimate is recalibrated to align with a lender’s internal default experience.

This model formulation operates naturally in the *log-odds* space—the canonical scale of logistic regression. The functional form of the calibration model is as follows:

- Intercept: $\beta_0 = -2.0$ (log-odds of default at the reference score)
- Slope: $\beta_1 = 1.5$ (increase in log-odds of default for each one-unit increase in score)

Because higher probabilities of default correspond to higher risk, β_1 is positive. At the reference score ($X = 0$), the default probability is $\frac{e^{-2.0}}{1+e^{-2.0}} \approx 0.12$ (12%). Each one-unit increase in score multiplies the odds of default by $e^{1.5} \approx 5$.

The data is generated such that the probability of default follows:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}} \quad (2.41)$$

Our goal is to recover these true parameters from the observed data using gradient descent. To incorporate an intercept term, we augment the design matrix $\mathbf{X}$ with a column of ones, so each observation has the form $\mathbf{x}_i = [1, x_i]$.

We initialize the parameters at $\beta_0 = 0$ and $\beta_1 = 0$, giving $\boldsymbol{\beta} = [0, 0]$. This initialization corresponds to a model that predicts $P(y = 1) = 0.5$ for all observations, regardless of their feature values—treating both outcomes as equally likely. From this starting point, gradient descent will iteratively adjust the parameters to incorporate the observed patterns in the data, moving toward the maximum likelihood estimates.

The gradient descent algorithm follows a straightforward iterative logic: compute predictions using the current parameters, measure the discrepancy between predictions and observations, and update the parameters to reduce this discrepancy. Each iteration moves the parameters in the direction that most improves the fit to the data, as measured by the log-likelihood. The process continues until the parameters stabilize at values that maximize the likelihood of observing the data. The mechanics of this procedure unfold in three steps.

Gradient Descent Steps

Step 1: Initial Prediction

The linear predictor is computed as $\mathbf{X}^\top \boldsymbol{\beta}$, which operates in log-odds space. We convert this to probabilities using the sigmoid (logistic) function:

$$\hat{p}_i = \frac{1}{1 + e^{-\mathbf{x}_i^\top \boldsymbol{\beta}}}.$$

Step 2: Calculate Residuals

For each observation, compute the residual^a:

$$r_i = \hat{p}_i - y_i.$$

For instance, if a defaulted loan ($y_i = 1$) is predicted with $\hat{p}_i = 0.3$, then $r_i = -0.7$. These residuals guide the next iteration toward parameters that minimize prediction error.

Step 3: Update Weights

Standard gradient descent updates parameters using first-order information:

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \alpha \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}),$$

where α is the learning rate. Gradient descent can be slow and requires careful tuning of α to ensure stable convergence.

^aIn practice, logistic regression maximizes the log-likelihood, which is equivalent to minimizing the negative log-likelihood—hence the use of the term “gradient descent”. If we instead defined the residual as $y_i - \hat{p}_i$, the update would correspond to gradient ascent. The descent formulation is more common, so we adopt it here.

Fisher Scoring: Adding Second-Order Information

While gradient descent is computationally inexpensive per iteration, modern implementations achieve faster overall convergence by reducing the number of iterations required by incorporating Fisher information, the second-order derivative information about the likelihood surface.

Fisher information equals the negative expected Hessian and can be expressed as:

$$\mathcal{I}(\boldsymbol{\beta}) = \mathbf{X}^\top \mathbf{W} \mathbf{X} \tag{2.42}$$

where $\mathbf{W}$ is a diagonal matrix with entries $w_i = \hat{p}_i(1 - \hat{p}_i)$. This $\hat{p}_i(1 - \hat{p}_i)$ term is the Bernoulli variance, which depends only on the predicted probability—not on whether the actual outcome is $y_i = 1$ or $y_i = 0$ ⁷. This creates a variance-covariance matrix that captures the curvature of the likelihood surface.

The Fisher scoring update becomes:

⁷An alternative approach is to use the outer product of gradients, known as the *Empirical Fisher Information*. This method relies solely on observed score vectors rather than the model-implied expectations. While both approaches yield identical coefficient estimates, the empirical version typically produces slightly different and often less precise standard errors.

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + \mathcal{I}(\boldsymbol{\beta}^{(t)})^{-1} \mathbf{X}^\top (\mathbf{y} - \hat{\mathbf{p}}) \quad (2.43)$$

This update rule⁸ transforms the optimization problem into *iteratively reweighted least squares* (IRLS), which converges much faster than standard gradient descent and requires no learning rate tuning.

Fisher scoring and IRLS are equivalent for logistic regression because the expected Fisher information (the variance of the MLE used in Fisher scoring) equals the observed information (used in Newton-Raphson) for canonical link functions like the logit link. This special property of the logistic model means both methods produce identical updates at each iteration. This approach forms the basis of most statistical software implementations, including Python’s `statsmodels`, R’s `glm()`, and SAS PROC LOGISTIC.

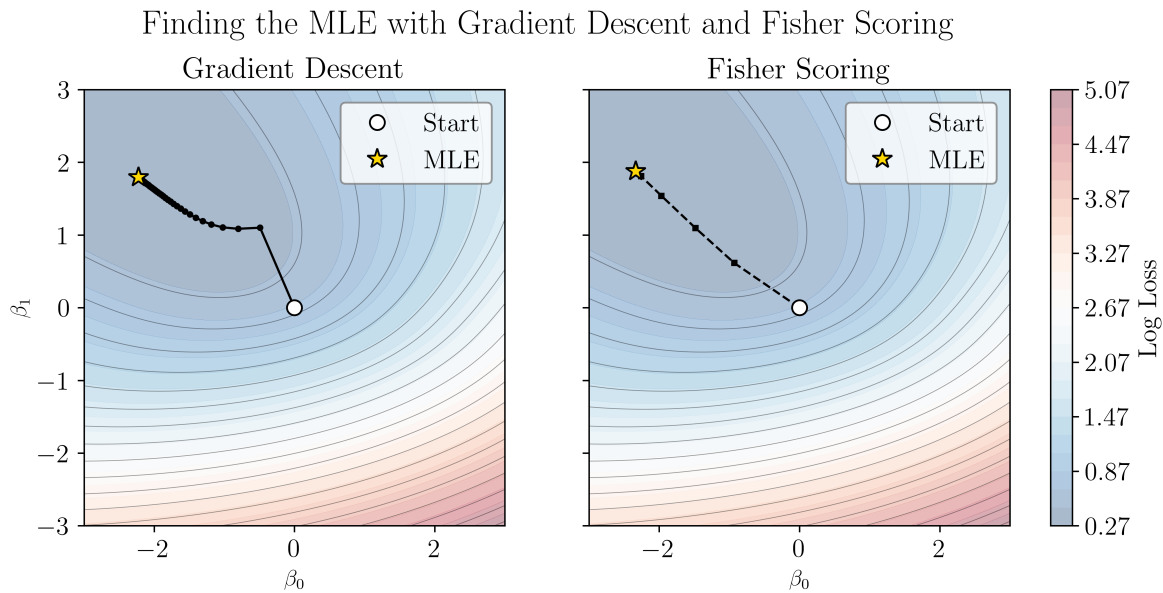


Figure 2.6: Comparison of optimization trajectories for Gradient Descent and Fisher Scoring. Each panel shows contour lines of the **log loss surface** over parameters β_0 (intercept) and β_1 (slope). Both algorithms begin at the origin $\boldsymbol{\beta} = [0, 0]$ (open circle) and converge to the maximum likelihood estimate (star). Fisher Scoring achieves faster convergence by taking curvature-informed quadratic steps, whereas Gradient Descent follows smaller, linear updates.

As illustrated in Figure 2.6, Fisher Scoring achieves faster convergence by taking curvature-informed quadratic steps on the **log loss surface**, which is proportional to the negative log-likelihood surface.

⁸This expression follows the standard maximum likelihood convention, where the log-likelihood is maximized directly. In contrast, gradient descent minimizes the negative log-likelihood, which reverses the sign of the update.

A note on Weight of Evidence (WOE) Logistic Regression

A combination of Weight of Evidence (WOE) and Logistic Regression became a staple in credit risk modeling. Yet, most applied practitioners view Weight of Evidence (WOE) as merely a feature encoding step, with logistic regression doing the “real” work. **But this is a misconception:** WOE transformations capture predictive power through likelihood ratios, while logistic regression adjusts for dependencies among features.

The mathematical reality: Under the naive Bayes assumption (features are independent given the class), WOE values can be seen as stand-alone univariate logistic regression models:

$$\log \frac{P(Y = 1|X)}{P(Y = 0|X)} = \log \frac{P(Y = 1)}{P(Y = 0)} + \sum_{j=1}^p \text{WOE}_j(x_j) \quad (2.44)$$

Each $\text{WOE}_j(x_j) = \log \frac{p(x_j|Y=1)}{p(x_j|Y=0)}$ is a likelihood ratio that quantifies how much evidence feature j provides for class 1 versus class 0^a.

Yet, in practice, features are often correlated. Spiegelhalter and Knill-Jones [22] found that in medical diagnostics, symptoms and test results are often correlated, leading to overconfident predictions when using raw WOE sums. Their solution was to use WOE values as inputs to logistic regression:

$$\log \frac{P(Y = 1|X)}{P(Y = 0|X)} = \beta_0 + \sum_{j=1}^p \beta_j \times \text{WOE}_j(x_j) \quad (2.45)$$

Here, the coefficients β_j serve as *adjustment factors* that account for feature dependencies. If features were truly independent, we would expect $\beta_j \approx 1$ for all j . Deviations from 1 indicate the degree to which the naive independence assumption is violated. **In essence**, WOE captures each feature’s univariate predictive signal through likelihood ratios, while logistic regression recalibrates these signals to prevent double-counting correlated evidence. **An ensemble perspective:** Each WOE value corresponds to a univariate logistic regression that captures the relationship between one predictor and the outcome, expressed in log-odds space. The final logistic regression then acts as a meta-learner that combines these univariate predictions:

Feature	Univariate model	Meta-learner
$x_1, \dots, x_p$	$\text{WOE}_1, \dots, \text{WOE}_p$	$\beta_0 + \sum_j \beta_j \text{WOE}_j$

Each univariate model contributes its “vote” (the WOE value), and the meta-learner (logistic regression) learns how much weight to give each vote, accounting for redundancy when features are correlated. This is why the method remains competitive with modern machine learning: it effectively combines strong univariate feature engineering with flexible multivariate calibration.

^aIn credit risk texts, the fraction is often inverted so that higher scores correspond to better credit quality, which reverses the sign of the resulting coefficients.

2.5 From Binary to Multinomial Logistic Regression

Up to this point, we have focused on binary outcomes—events that can occur or not occur, such as default vs. no default or fraud vs. no fraud. Logistic regression in this setting models the log-odds of one outcome relative to the other.

However, many real-world problems involve more than two possible outcomes: customers may be classified into a number of segments, transactions may belong to several fraud types—such as first-party fraud or account takeover—and a loan may end up in one of several delinquency states—a topic central to collections scoring.

A note on Ordinal Outcomes

When outcomes have a natural ordering (e.g., credit ratings from AAA to D, or delinquency buckets like current, 30+ days past due, 60+ days past due, 90+ days past due), *ordinal logistic regression* models can be more appropriate than standard multinomial models. These models, particularly common in low-default portfolio modeling, respect the ordering by modeling cumulative probabilities rather than treating categories as unordered. While we focus on multinomial logistic regression in this chapter, the fundamental principles—maximum likelihood estimation, log-odds transformations, and the connection to WOE—extend naturally to ordinal models.

Beyond credit risk and finance, the same principle underlies state-of-the-art machine learning systems. For instance, large language models predict the probability distribution over thousands of possible next tokens, which is an inherently multinomial classification problem. The logic of logistic regression extends naturally to these settings.

This generalization from the binomial to the multinomial was already implicit in early probability theory. Ronald Fisher, with his characteristic insight, recognized that the binomial distribution—formalized by Jacob Bernoulli and studied for over two centuries—was merely a special case of a broader family: the multinomial.

As Fisher [11] wrote with characteristic precision:

“The binomial distribution... was found by Bernoulli... that if the probability of an event occurring were p and the probability of it not occurring were $q = 1 - p$, then if a random sample of n trials were taken, the frequencies with which the event occurred 0, 1, 2, ..., n times were given by the expansion of the binomial $(q + p)^n$. This rule is a particular case of a more general theorem dealing with cases in which not only a simple alternative is considered, but in which the event may happen in s ways with probabilities $p_1, p_2, \dots, p_s$.”

This insight—recognizing the binomial as a special case of the multinomial distribution—directly motivates the extension from binary logistic regression to multinomial logistic regression.

Binary vs. Multinomial Logistic Models

Binary Case (Bernoulli):

- Two outcomes: success (p) and failure ($1 - p$)
- One linear predictor: $\eta = x^\top \beta$
- Model: $\text{logit}(p) = \eta$

Multinomial Case:

- K possible mutually exclusive outcomes with probabilities $p_1, \dots, p_K$
- Probabilities satisfy $\sum_{k=1}^K p_k = 1$
- Parameterization via the *softmax* function:

$$p_k(x) = \frac{\exp(x^\top \beta_k)}{\sum_{j=1}^K \exp(x^\top \beta_j)}.$$

- Two equivalent formulations:
 - **Statistics approach:** Fix one class (e.g., $\beta_K = 0$), yielding $K - 1$ parameter vectors for identifiability.
 - **Machine learning approach:** Estimate all K coefficient vectors directly using the unconstrained softmax form, producing a coefficient matrix of size $(p \times K)$, where p is the number of predictors (including the intercept).

Both parameterizations yield identical class probabilities. The constrained $(K - 1)$ form emphasizes identifiability, while the full- K softmax form is numerically stable and widely used in machine learning frameworks.

Softmax as Generalization of Binary Logistic Regression

Binary logistic regression can be viewed as multinomial regression with $K - 2$ classes. The key observation is that we only need to model $K - 1$ probabilities explicitly, since the K th probability is determined by the constraint that all probabilities sum to 1.

The general softmax formulation for K classes is:

$$P(Y = k | \mathbf{X}) = \frac{\exp(\mathbf{X}\beta_k)}{\sum_{j=1}^K \exp(\mathbf{X}\beta_j)} \quad (2.46)$$

For binary classification ($K=2$), setting $\beta_0 = \mathbf{0}$ (reference category) recovers the standard logistic regression formulation:

$$P(Y = 1|\mathbf{x}) = \frac{\exp(\beta_1^T \mathbf{x})}{\exp(\mathbf{0}) + \exp(\beta_1^T \mathbf{x})} = \frac{\exp(\beta^T \mathbf{x})}{1 + \exp(\beta^T \mathbf{x})} \quad (2.47)$$

$$P(Y = 0|\mathbf{x}) = \frac{\exp(\mathbf{0})}{\exp(\mathbf{0}) + \exp(\beta_1^T \mathbf{x})} = \frac{1}{1 + \exp(\beta^T \mathbf{x})} \quad (2.48)$$

What Happens If We Fit Softmax with Two Classes?

An interesting theoretical question arises: what if we fit a binary softmax model *without* constraining one weight vector to zero?

In this case, we use two separate weight vectors, $\mathbf{w}_0$ and $\mathbf{w}_1$, one for each class, and optimize them jointly via maximum likelihood.

Below we show the resulting coefficient matrix from such a fit on our earlier example in Table 2.2:

$$\mathbf{W} = [\mathbf{w}_0 \quad \mathbf{w}_1] = \begin{bmatrix} 1.6094 & -1.6094 \\ -1.4979 & 1.4979 \end{bmatrix} \quad \begin{array}{l} \mathbf{w}_0 \text{ for class 0} \\ \mathbf{w}_1 \text{ for class 1} \end{array}$$

$$\mathbf{w}_0 = \begin{bmatrix} 1.6094 \\ -1.4979 \end{bmatrix}, \quad \mathbf{w}_1 = \begin{bmatrix} -1.6094 \\ 1.4979 \end{bmatrix}, \quad \mathbf{w}_0 = -\mathbf{w}_1.$$

Notice that the sum of corresponding coefficients equals zero, confirming the model's symmetry. This property implies that the coefficients for one class are the negatives (or complements) of those for the remaining classes, ensuring consistent log-odds relationships across all outcomes.

The softmax probability for class 1 is:

$$p_1(\mathbf{x}) = \frac{\exp(\mathbf{w}_1^T \mathbf{x})}{\exp(\mathbf{w}_1^T \mathbf{x}) + \exp(\mathbf{w}_0^T \mathbf{x})} = 0.444. \quad (2.49)$$

This matches our $P(Y = 1)$ from the standard binary logistic regression formulation.

Hence, the relationship between softmax and binary logistic regression coefficients can be expressed as:

$$\begin{aligned} \mathbf{w}_1 &= \frac{1}{2} \mathbf{w}_{\text{logistic}}, \\ \mathbf{w}_0 &= -\frac{1}{2} \mathbf{w}_{\text{logistic}}. \end{aligned} \quad (2.50)$$

Multinomial Maximum Likelihood Estimation

The multinomial log-likelihood extends the binary case using the 1-of-K coding scheme, where the target vector for observation i belonging to class k is a binary vector with all elements zero except element k , which equals one.

The likelihood function is:

$$p(\mathbf{T}|\mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{i=1}^n \prod_{k=1}^K p_{ik}^{t_{ik}} \quad (2.51)$$

where $p_{ik} = P(Y_i = k|\mathbf{x}_i)$ and t_{ik} is the target indicator.

Taking the negative logarithm gives the **cross-entropy error function**:

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = - \sum_{i=1}^n \sum_{k=1}^K t_{ik} \log(p_{ik}) \quad (2.52)$$

where:

$$p_{ik} = \frac{\exp(\mathbf{w}_k^T \mathbf{x}_i)}{\sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x}_i)} \quad (2.53)$$

Solving Multinomial Logistic Regression By Hand

In this section, we will work with the following data extended to support the classification of Days Past Due (DPD) and Unlikely to Pay (UTP) defaults for a consumer lending product.

Evidence (X)	DPD Default ($y = 2$)	UTP Default ($y = 1$)	No Default ($y = 0$)	Row Total
Delinquent > 5 days	50	30	100	180
Delinquent $\leq$ 5 days	5	15	500	520
Column Total	55	45	600	700

Table 2.3: Multiclass Credit Risk Dataset: DPD vs UTP Default Types

We can calculate the coefficients using the following approach, with "No Default" ($y = 0$) as the reference category.

Step 1: Calculate Group-Specific Probabilities

For each class comparison, we only consider counts for that class vs the reference (No Default). For example, when comparing UTP to No Default in the high delinquency group, we use only the 30 UTP defaults and 100 non-defaults (ignoring the 50 DPD defaults), giving us a denominator of $30 + 100 = 130$.

Group	UTP (y=1) vs No (y=0)	DPD (y=2) vs No (y=0)
High delinquency ($x = 1$)	$\frac{30}{30+100} = \frac{30}{130} = 0.2308$	$\frac{50}{50+100} = \frac{50}{150} = 0.3333$
Low delinquency ($x = 0$)	$\frac{15}{15+500} = \frac{15}{515} = 0.0291$	$\frac{5}{5+500} = \frac{5}{505} = 0.0099$

Step 2: Calculate Log-Odds Relative to Reference Category

Convert probabilities to log-odds using $\log(p/(1-p))$. For example, for UTP in the high delinquency group: $1 - 0.2308 = 0.7692$, so the log-odds is $\log(0.2308/0.7692) = -1.204$.

Group	UTP (y=1) vs No (y=0)	DPD (y=2) vs No (y=0)
High delinquency ($x = 1$)	$\log\left(\frac{0.2308}{1-0.2308}\right) = -1.204$	$\log\left(\frac{0.3333}{1-0.3333}\right) = -0.693$
Low delinquency ($x = 0$)	$\log\left(\frac{0.0291}{1-0.0291}\right) = -3.507$	$\log\left(\frac{0.0099}{1-0.0099}\right) = -4.605$

Step 3: Calculate Coefficients

	UTP Default (y=1)	DPD Default (y=2)
Intercept (β_0)	-3.5066	-4.6052
Slope (β_1)	$-1.2040 - (-3.5066) = 2.3026$	$-0.6931 - (-4.6052) = 3.9120$

Step 4: Calculate Standard Errors

	UTP Default (y=1)	DPD Default (y=2)
SE(Intercept)	$\sqrt{\frac{1}{15} + \frac{1}{500}} = 0.262$	$\sqrt{\frac{1}{5} + \frac{1}{500}} = 0.449$
SE(Slope)	$\sqrt{\frac{1}{30} + \frac{1}{100} + \frac{1}{15} + \frac{1}{500}} = 0.335$	$\sqrt{\frac{1}{50} + \frac{1}{100} + \frac{1}{5} + \frac{1}{500}} = 0.482$

Wrapping it all together, we can reconstruct the logistic regression output for each class.⁹

UTP Default (y=1) vs No Default (y=0):

Parameter	Estimate	Std. Error	Wald Stat	P-value	Lower CI	Upper CI
Intercept ($\beta_{0,1}$)	-3.507	0.262	-13.38	< 0.001	-4.020	-2.993
Slope ($\beta_{1,1}$)	2.303	0.335	6.88	< 0.001	1.647	2.959

DPD Default (y=2) vs No Default (y=0):

Parameter	Estimate	Std. Error	Wald Stat	P-value	Lower CI	Upper CI
Intercept ($\beta_{0,2}$)	-4.605	0.449	-10.25	< 0.001	-5.486	-3.724
Slope ($\beta_{1,2}$)	3.912	0.482	8.12	< 0.001	2.968	4.856

While the example above demonstrates that the parameters of a multinomial logistic regression can be derived in closed form for simple grouped data, this approach does not generalize once multiple predictors or continuous variables are introduced. In such cases, parameter estimation must be performed iteratively.

⁹For brevity, we didn't include the calculation of confidence intervals above, but they follow the standard formula: $CI = \hat{\beta} \pm 1.96 \times SE(\hat{\beta})$. For example, UTP Intercept: $-3.507 \pm 1.96 \times 0.262 = [-4.020, -2.993]$.

Conceptually, the optimization procedure mirrors the binary case discussed earlier. Depending on the chosen formulation, we can either adopt the $K - 1$ baseline approach described above—where one class serves as the reference and only $K - 1$ coefficient vectors are estimated—or estimate parameters for all classes simultaneously using the *softmax* representation. In both cases, the underlying principles remain identical to those of binary logistic regression—the difference lies only in the dimensionality of the coefficient matrix.

For example, in the softmax formulation, the model learns a coefficient matrix with one vector per class:

$$\mathbf{W} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_K \\ | & | & \cdots & | \end{bmatrix}, \quad \text{where } \mathbf{w}_k = \begin{bmatrix} \beta_{0k} \\ \beta_{1k} \\ \vdots \\ \beta_{pk} \end{bmatrix}.$$

Each column $\mathbf{w}_k$ corresponds to the coefficient vector for class k , and the full matrix $\mathbf{W} \in \mathbb{R}^{(p+1) \times K}$.

Implementation Note

Since the iterative estimation process follows directly from the binary case—with the target represented in one-hot form across all K classes, we do not elaborate on it further here. Note that this implementation estimates parameters for all K classes simultaneously, with the softmax normalization ensuring the probabilities sum to one, rather than using a reference category approach, which would drop the reference class column from the one-hot encoding and estimate only $K - 1$ coefficient vectors.

Multiclass Weight of Evidence

The Weight of Evidence (WOE) framework we discussed in the earlier section when talking about logistic regression with two classes extends naturally to multiclass problems. As I.J. Good [13] noted, the same odds-form Bayes formula applies whether the hypotheses are simple (comparing specific classes pairwise) or composite (comparing one class against its alternative). We demonstrate both approaches using our DPD vs UTP default example in Table 2.3.

Approach 1: Simple Hypotheses (Against Reference Class)

This approach compares each class directly to a reference class (No Default), similar to multinomial logistic regression.

Step 1: Calculate Conditional Probabilities

For each evidence category, we calculate $P(X|Y)$:

Evidence (X)	DPD Default P(X Y=2)	UTP Default P(X Y=1)	No Default P(X Y=0)
Delinquent > 5 days	0.9091	0.6667	0.1667
Delinquent ≤ 5 days	0.0909	0.3333	0.8333
Column Sum	1.0000	1.0000	1.0000

Table 2.4: Conditional Probabilities P(X|Y)

Step 2: Calculate Likelihood Ratios and WOE

For **Delinquent > 5 days**, comparing each class to No Default¹⁰:

$$LR_{2|0} = \frac{P(X|Y=2)}{P(X|Y=0)} = \frac{0.9091}{0.1667} = 5.4545, \quad \text{WOE}_{\text{DPD}} = \log(5.4545) = 1.6958 \quad (2.54)$$

$$LR_{1|0} = \frac{P(X|Y=1)}{P(X|Y=0)} = \frac{0.6667}{0.1667} = 4.0000, \quad \text{WOE}_{\text{UTP}} = \log(4.0000) = 1.3863 \quad (2.55)$$

Step 3: Calculate Posterior Probabilities

Starting with prior odds:

$$\text{Prior Odds}_{\text{DPD vs No Default}} = \frac{P(Y=2)}{P(Y=0)} = \frac{55}{600} = 0.0917 \quad (2.56)$$

$$\text{Prior Odds}_{\text{UTP vs No Default}} = \frac{P(Y=1)}{P(Y=0)} = \frac{45}{600} = 0.0750 \quad (2.57)$$

Applying the likelihood ratios:

$$\text{Posterior Odds}_{\text{DPD}} = 0.0917 \times 5.4545 = 0.4999 \quad (2.58)$$

$$\text{Posterior Odds}_{\text{UTP}} = 0.0750 \times 4.0000 = 0.3000 \quad (2.59)$$

Converting to probabilities using the multinomial softmax formula:

$$P(\text{DPD}|\text{high delinquency}) = \frac{0.4999}{1 + 0.4999 + 0.3000} = \frac{0.4999}{1.7999} = 0.2778 \quad (2.60)$$

$$P(\text{UTP}|\text{high delinquency}) = \frac{0.3000}{1.7999} = 0.1667 \quad (2.61)$$

$$P(\text{No Default}|\text{high delinquency}) = \frac{1}{1.7999} = 0.5556 \quad (2.62)$$

¹⁰**Note:** The likelihood ratio for the reference class (No Default) is $LR_{0|0} = \frac{P(X|Y=0)}{P(X|Y=0)} = 1$, so it contributes no information and is omitted from calculations. This is why we only need to compute likelihood ratios for the non-reference classes.

Approach 2: Simple vs Composite Hypotheses (Against Alternative)

This approach compares each class against all others combined, useful when focusing on a specific outcome of interest. We call the negation of a class of interest a *composite hypothesis*, because it is composed of several simple hypotheses.

Connection to Approach 1: Notice that the posterior probabilities we calculated in Approach 1 (see Step 3 on page 44) are exactly the values we'll use as our starting point here. Specifically, for high delinquency:

- $P(\text{DPD}|X = 1) = 0.2778$
- $P(\text{UTP}|X = 1) = 0.1667$
- $P(\text{No Default}|X = 1) = 0.5556$

These same probabilities appear in the table that follows.

To calculate WOE with the second approach, we present the data in terms of the posterior probabilities $P(Y|X)$:

Evidence (X)	DPD Default P(Y=2 X)	UTP Default P(Y=1 X)	No Default P(Y=0 X)
Delinquent > 5 days	0.2778	0.1667	0.5556
Delinquent ≤ 5 days	0.0096	0.0288	0.9615
Row Sum	1.0000	1.0000	1.0000

Table 2.5: Posterior Probabilities $P(Y|X)$

For **Delinquent > 5 days**, comparing each class to all others:

Step 1: Prior and Posterior Odds

$$\text{Prior Odds}_{\text{DPD vs } \neg \text{DPD}} = \frac{55}{645} = 0.0853, \quad \text{Posterior Odds} = \frac{50}{130} = 0.3846 \quad (2.63)$$

$$\text{Prior Odds}_{\text{UTP vs } \neg \text{UTP}} = \frac{45}{655} = 0.0687, \quad \text{Posterior Odds} = \frac{30}{150} = 0.2000 \quad (2.64)$$

$$\text{Prior Odds}_{\text{No Default vs } \neg \text{No Default}} = \frac{600}{100} = 6.0000, \quad \text{Posterior Odds} = \frac{100}{80} = 1.2500 \quad (2.65)$$

Step 2: Likelihood Ratios

To calculate likelihood ratios, we divide posterior odds by prior odds:

$$LR_{2|-2} = \frac{\text{Posterior Odds}_{\text{DPD}}}{\text{Prior Odds}_{\text{DPD}}} = \frac{0.3846}{0.0853} = 4.5105 \quad (2.66)$$

$$LR_{1|-1} = \frac{\text{Posterior Odds}_{\text{UTP}}}{\text{Prior Odds}_{\text{UTP}}} = \frac{0.2000}{0.0687} = 2.9111 \quad (2.67)$$

$$LR_{0|-0} = \frac{\text{Posterior Odds}_{\text{No Default}}}{\text{Prior Odds}_{\text{No Default}}} = \frac{1.2500}{6.0000} = 0.2083 \quad (2.68)$$

Step 3: Weight of Evidence (WOE)

The WOE is simply the log of the likelihood ratio:

$$\text{WOE}_{\text{DPD}} = \log(LR_{2|-2}) = \log(4.5105) = 1.5063 \quad (2.69)$$

$$\text{WOE}_{\text{UTP}} = \log(LR_{1|-1}) = \log(2.9111) = 1.0686 \quad (2.70)$$

$$\text{WOE}_{\text{No Default}} = \log(LR_{0|-0}) = \log(0.2083) = -1.5686 \quad (2.71)$$

These WOE values quantify the evidence that each delinquency status provides for each outcome class.

Notice that $\text{WOE}_{\text{No Default}} = -1.5686$ is the negative of the binary WOE for default in the high delinquency group ($\text{WOE}_{x=1} = 1.5686$) we saw in Box 2.3. This makes intuitive sense: when we combine DPD and UTP into a single “Default” category, the evidence against default (favoring No Default) is the opposite of the evidence for default.

Step 4: From WOE to Probabilities

We can derive the posterior probabilities using the relationship: Posterior Log-Odds = Prior Log-Odds + WOE

For DPD:

$$\text{Prior Log-Odds}_{\text{DPD}} = \log(0.0853) = -2.4617 \quad (2.72)$$

$$\text{WOE}_{\text{DPD}} = 1.5063 \quad (2.73)$$

$$\text{Posterior Log-Odds}_{\text{DPD}} = -2.4617 + 1.5063 = -0.9554 \quad (2.74)$$

$$\text{Posterior Odds}_{\text{DPD}} = e^{-0.9554} = 0.3846 \quad (2.75)$$

$$P(\text{DPD}|\text{high delinquency}) = \frac{0.3846}{1 + 0.3846} = 0.2778 \quad (2.76)$$

For UTP:

$$\text{Prior Log-Odds}_{\text{UTP}} = \log(0.0687) = -2.6779 \quad (2.77)$$

$$\text{WOE}_{\text{UTP}} = 1.0686 \quad (2.78)$$

$$\text{Posterior Log-Odds}_{\text{UTP}} = -2.6779 + 1.0686 = -1.6093 \quad (2.79)$$

$$\text{Posterior Odds}_{\text{UTP}} = e^{-1.6093} = 0.2000 \quad (2.80)$$

$$P(\text{UTP}|\text{high delinquency}) = \frac{0.2000}{1 + 0.2000} = 0.1667 \quad (2.81)$$

For No Default:

$$\text{Prior Log-Odds}_{\text{No Default}} = \log(6.0000) = 1.7918 \quad (2.82)$$

$$\text{WOE}_{\text{No Default}} = -1.5686 \quad (2.83)$$

$$\text{Posterior Log-Odds}_{\text{No Default}} = 1.7918 + (-1.5686) = 0.2232 \quad (2.84)$$

$$\text{Posterior Odds}_{\text{No Default}} = e^{0.2232} = 1.2500 \quad (2.85)$$

$$P(\text{No Default} | \text{high delinquency}) = \frac{1.2500}{1 + 1.2500} = 0.5556 \quad (2.86)$$

This demonstrates how WOE directly quantifies the shift from prior to posterior odds in log-space for each class.

This concludes our investigation into the extension of logistic regression from binary to multinomial settings. We have seen that the same fundamental ideas—log-odds, likelihood ratios, and weight of evidence—extend seamlessly when the outcome involves multiple categories. The softmax formulation generalizes the logistic link, the maximum likelihood framework remains unchanged, and the familiar interpretability of odds and log-odds carries through to higher dimensions. Whether implemented through the $K - 1$ baseline form or the full softmax representation, the multinomial model preserves the probabilistic rigor and transparency that make logistic regression such a cornerstone of statistical learning.

The Weight of Evidence formulation further demonstrates how these principles can be expressed in an intuitive and interpretable way, bridging traditional credit risk modeling with modern machine learning algorithms. In the chapters that follow, we will build upon these foundations to explore regularization, calibration, and practical implementation strategies for large-scale predictive modeling.

2.6 Logistic Regression Inference: Quantifying Uncertainty

Having established the foundations of logistic regression from binary to multinomial cases, we now turn to a critical practical question: *how do we quantify the uncertainty in our predictions?*

Consider a credit risk model that predicts a 15% probability of default for a loan applicant. Should we treat this as a precise estimate, or could the true probability be anywhere from 10% to 20%? This distinction matters enormously: regulatory capital requirements, pricing decisions, and risk management strategies all depend on understanding not just the point estimate, but the range of plausible values.

Interpreting Coverage for Probability Intervals

Before we proceed to the discussion of evaluating predictive intervals for probabilities, it is important to clarify **what “coverage” means**.

A probability prediction interval such as $[\hat{p}_L, \hat{p}_U]$ is not meant to contain the binary outcome $y_i \in \{0, 1\}$, but rather to provide valid uncertainty quantification for the *true conditional probability* $p(x_i) = \Pr(Y = 1 \mid X = x_i)$. Since individual-level true probabilities are unobservable, coverage is assessed through *empirical event rates*:

- **Binary coverage** — the proportion of cases where $y_i \in [\hat{p}_L, \hat{p}_U]$ — is usually near 0% or 100%, depending on whether intervals are narrow or span most of $[0, 1]$. It is *not* an appropriate metric for probability intervals.
- **Conditional coverage** — the key validity criterion — requires that within groups of similar predictions, the observed event rate falls within the predicted interval approximately $(1 - \alpha)$ of the time. Formally: $\Pr(p(X) \in [\hat{p}_L(X), \hat{p}_U(X)] \mid X \in \mathcal{S}) \geq 1 - \alpha$ for subsets $\mathcal{S}$ of the feature space.
- **Marginal coverage** — a weaker but still valid guarantee — requires that averaged over the test distribution, intervals contain true probabilities with frequency $(1 - \alpha)$: $\Pr(p(X) \in [\hat{p}_L(X), \hat{p}_U(X)]) \geq 1 - \alpha$.

Note on standard conformal prediction: Classical conformal prediction for binary classification produces *prediction sets* such as {“default”}, {“not_default”}, or {“default”, “not_default”} with coverage guarantees for the true class label. In credit risk, however, we need *probability intervals* that quantify uncertainty in $\hat{p}_i$ itself, not just class membership. This requires conformalized approaches that adapt the conformal framework to continuous probability estimates. Intuitively, valid coverage for probability intervals answers:

“When I predict uncertainty bounds $[\hat{p}_L, \hat{p}_U]$ for groups of similar cases, how often does the observed event rate in those groups fall inside my stated interval?”

This distinction explains why properly calibrated methods can yield wide intervals with zero binary coverage yet still provide valid uncertainty quantification—they are calibrated to cover *true underlying probabilities*, not discrete outcomes.

While logistic regression provides point estimates of probabilities, real-world applications—particularly in credit risk and regulatory compliance—require understanding the confidence we can place in these estimates. **This section introduces two complementary approaches** to quantifying this uncertainty:

- **Wald-type confidence intervals:** The classical approach, derived from the estimated covariance matrix of the coefficients. This method leverages the asymptotic normality of maximum likelihood estimators [14].
- **Conformalized prediction:** A modern, distribution-free approach that uses empirical residuals to construct probability intervals with guaranteed coverage properties.

Why two approaches? Wald intervals are computationally efficient and widely implemented, but rely on asymptotic assumptions that may not hold in small samples. Conformalized prediction

offers finite-sample validity without parametric assumptions, at the cost of requiring a separate calibration dataset. Understanding both methods equips practitioners to choose the right tool for their specific context.

Wald-Type Confidence Intervals for Logistic Regression

The most widely used method for inference in logistic regression relies on the asymptotic normality of the maximum likelihood estimators (MLEs). Under regularity conditions, the estimated coefficients $\hat{\beta}$ are approximately normal with covariance matrix equal to the inverse of the observed information matrix.

The key intuition: We treat the model parameters as random variables centered around their estimated values, with uncertainty captured by the curvature of the likelihood surface. A sharply peaked likelihood (high curvature) indicates we have strong information about the parameters, leading to narrow confidence intervals. A flat likelihood (low curvature) indicates greater uncertainty and wider intervals.

In logistic regression, the model updates during estimation are given by:

$$\beta^{(t+1)} = \beta^{(t)} + [\mathbf{X}^\top \mathbf{W}^{(t)} \mathbf{X}]^{-1} \mathbf{X}^\top (\mathbf{y} - \mathbf{p}^{(t)}), \quad (2.87)$$

where $\mathbf{W}^{(t)}$ is a diagonal matrix with elements $w_i = \hat{p}_i^{(t)}(1 - \hat{p}_i^{(t)})$. At convergence, the Fisher Information Matrix is:

$$\mathcal{I}(\beta) = \mathbf{X}^\top \mathbf{W} \mathbf{X}, \quad (2.88)$$

and the estimated covariance matrix of the coefficient estimates is its inverse:

$$\text{Cov}(\hat{\beta}) = \mathcal{I}(\beta)^{-1} = [\mathbf{X}^\top \mathbf{W} \mathbf{X}]^{-1}. \quad (2.89)$$

This inverse relationship reflects a fundamental principle: the more information we have about the parameters (higher Fisher Information), the lower the variance of our estimates.

In practice, this covariance matrix is automatically computed by statistical software during model fitting. The diagonal elements give the variances of individual coefficients, while off-diagonal elements capture correlations between coefficient estimates.

From coefficients to predictions: To construct confidence intervals for a new prediction, we need to propagate the uncertainty in $\hat{\beta}$ through to the predicted probability. This involves two steps: first, quantifying uncertainty in the logit (linear predictor), then transforming to the probability scale.

For a new observation $\mathbf{x}_0$, the estimated **logit** (linear predictor) is:

$$\hat{g}_0 = \mathbf{x}_0^\top \hat{\beta}, \quad (2.90)$$

and the corresponding predicted probability is:

$$\hat{p}_0 = \frac{1}{1 + e^{-\hat{g}_0}}. \quad (2.91)$$

The **variance of the estimated logit** is given by:

$$\widehat{\text{Var}}(\hat{g}_0) = \mathbf{x}_0^\top \mathcal{I}(\boldsymbol{\beta})^{-1} \mathbf{x}_0, \quad (2.92)$$

and the corresponding standard error is:

$$\text{SE}(\hat{g}_0) = \sqrt{\widehat{\text{Var}}(\hat{g}_0)}. \quad (2.93)$$

Using the asymptotic normality assumption, the $(1 - \alpha)$ **confidence interval** for the logit is:

$$\hat{g}_0 \pm z_{1-\alpha/2} \text{SE}(\hat{g}_0), \quad (2.94)$$

and the corresponding confidence interval for the probability is obtained by transforming back through the logistic function:

$$\hat{p}_L = \frac{1}{1 + \exp[-(\hat{g}_0 - z_{1-\alpha/2} \text{SE}(\hat{g}_0))]}, \quad (2.95)$$

$$\hat{p}_U = \frac{1}{1 + \exp[-(\hat{g}_0 + z_{1-\alpha/2} \text{SE}(\hat{g}_0))]}. \quad (2.96)$$

In summary, these intervals provide a parametric measure of uncertainty under the assumption that the MLE behaves approximately normally in large samples. They form the basis of what are commonly known as Wald-type confidence intervals, implemented by default in most statistical software packages (e.g., R's `predict.glm()` with `se.fit = TRUE`, Python's `statsmodels.GLM.get_prediction()`, and SAS's `PROC LOGISTIC`).

Practical considerations: Wald intervals work well when sample sizes are large and the model is well-specified. However, with very large samples, intervals can become narrow and may be overly optimistic, failing to account for model misspecification or systematic biases. Additionally, they can be unreliable in small samples or when predicted probabilities are near 0 or 1 (where the logistic transformation is highly nonlinear). **This motivates** the alternative approach we explore next: conformalized prediction.

Conformalized Prediction via Pearson Residuals

While Wald intervals rely on asymptotic normality and model correctness, conformalized prediction provides an alternative, *distribution-free* approach. Rather than relying on theoretical assumptions, it uses empirical nonconformity scores¹¹ from a calibration dataset to construct probability intervals that achieve valid coverage in finite samples.

The fundamental shift: Instead of asking "what does theory tell us about parameter uncertainty?", conformalized prediction asks "how wrong was our model on past data, and how can we use that to quantify future uncertainty?" This shifts the focus from parametric inference to data-driven calibration of uncertainty.

The approach in three steps:

¹¹Nonconformity scores to gauge the alignment between predictions and the training data distribution. Conformal prediction uses "nonconformity measures" to assess how well new observations fit with previous observations. We call it nonconformity because it measures how different or "nonconforming" the new observation is, compared to the training or calibration sets [16]. Please see Box 2.6 to understand the logic of using this specific measure.

1. Fit the model on training data
2. Calculate nonconformity scores on a separate calibration set
3. Use the distribution of these nonconformity scores to construct probability intervals (confidence bands) for new observations

We will use the **Pearson residual** [17] as our nonconformity score:

$$r_i = \frac{y_i - \hat{p}_i}{\sqrt{\hat{p}_i(1 - \hat{p}_i)}}, \quad (2.97)$$

where $y_i \in \{0, 1\}$ and $\hat{p}_i$ is the predicted probability.

Why This Form?

This choice follows the principle of *studentized residuals* in conformal prediction [23]. The general form of studentized residuals is:

$$R_i = \frac{|Y_i - \hat{f}_{n_1}(X_i)|}{\hat{\sigma}_{n_1}(X_i)}, \quad i \in D_2,$$

where $\hat{f}_{n_1}(X_i)$ is the point predictor and $\hat{\sigma}_{n_1}(X_i)$ is the spread predictor (estimated standard deviation) fitted on training set D_1 and evaluated on calibration set D_2 . The key insight is to normalize the raw prediction error $|y_i - \hat{p}_i|$ by a "spread predictor" that estimates the variability at each point. For binary outcomes, the natural spread predictor is $\sqrt{\hat{p}_i(1 - \hat{p}_i)}$, which is the standard deviation of a Bernoulli random variable. This normalization makes residuals comparable across different predicted probabilities: a prediction error of 0.1 at $\hat{p} = 0.5$ (high variance region) is treated differently than the same error at $\hat{p} = 0.95$ (low variance region). The resulting intervals adapt their width according to the local uncertainty.

Let $q_{1-\alpha}$ denote the $(1 - \alpha)$ quantile of the absolute residuals $|r_i|$ from the calibration set. The resulting **conformalized probability interval** (confidence band) for a new observation is:

$$[\hat{p}_L, \hat{p}_U] = \left[\max \left(0, \hat{p} - q_{1-\alpha} \sqrt{\hat{p}(1 - \hat{p})} \right), \min \left(1, \hat{p} + q_{1-\alpha} \sqrt{\hat{p}(1 - \hat{p})} \right) \right]. \quad (2.98)$$

The empirical coverage of these intervals is:

$$\text{Coverage} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}(y_i \in [\hat{p}_{L,i}, \hat{p}_{U,i}]), \quad (2.99)$$

which should approximate $(1 - \alpha)$ if the calibration is adequate. Here, $\mathbb{1}(\cdot)$ equals 1 if the true value lies within the predicted interval and 0 otherwise.

Logit-Based Variant

An alternative formulation applies the conformalized adjustment on the *logit* (log-odds) scale rather than directly on probabilities.

Starting from the Pearson residual $r_i = (y_i - \hat{p}_i) / \sqrt{\hat{p}_i(1 - \hat{p}_i)}$, the nonconformity score is computed as:

$$s_i = \left| \frac{r_i}{\sqrt{\hat{p}_i(1 - \hat{p}_i)}} \right| = \left| \frac{y_i - \hat{p}_i}{\hat{p}_i(1 - \hat{p}_i)} \right|.$$

Since $\hat{p}_i(1 - \hat{p}_i)$ is the derivative of the logistic function, s_i represents the local linear approximation of the logit shift needed to align predictions with observed outcomes.

After computing the calibration quantile $q_{1-\alpha} = \text{Quantile}_{1-\alpha}(\{s_i\}_{i \in \mathcal{I}_{\text{cal}}})$, we construct symmetric intervals on the logit scale:

$$\eta_i = \text{logit}(\hat{p}_i) = \log\left(\frac{\hat{p}_i}{1 - \hat{p}_i}\right), \quad [\eta_L, \eta_U] = [\eta_i - q_{1-\alpha}, \eta_i + q_{1-\alpha}].$$

We then transform back to the probability scale:

$$[\hat{p}_L, \hat{p}_U] = [\sigma(\eta_L), \sigma(\eta_U)], \quad \text{where} \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

This **logit-based conformalized** method produces intervals that respect the natural geometry of the logistic model and automatically remain within $[0, 1]$ without clipping.

The intervals are wider near $\hat{p}_i = 0.5$ and narrower near the boundaries, reflecting the nonlinear structure of the logit scale.

Like the Pearson-based approach, this method provides valid conformalized coverage guarantees under the exchangeability assumption (see Figure 2.7).

In essence, conformalized prediction turns residual variability into predictive uncertainty, offering valid coverage guarantees without relying on asymptotic or parametric assumptions.

Terminology note: In classification, we construct *probability intervals* or *confidence bands* for $\hat{p}_i$, not *prediction intervals* for the binary outcome $y_i \in \{0, 1\}$. Prediction intervals are typically reserved for continuous regression outcomes, where we predict a range for the actual value. Here, we quantify uncertainty about the *probability estimate* itself.

Why Pearson residuals? The Pearson residual standardizes the prediction error by the standard deviation of a Bernoulli random variable, making residuals comparable across different predicted probabilities. A prediction of 0.5 with an error of 0.1 is treated differently than a prediction of 0.95 with the same error, because the latter represents a more surprising deviation.

Practical advantages: Conformalized prediction provides finite-sample validity guarantees and works with any model (not just logistic regression). The main cost is requiring a separate calibration dataset, which reduces the effective training sample size. In applications where data is abundant and coverage guarantees are critical—such as regulatory compliance—this trade-off is often worthwhile.

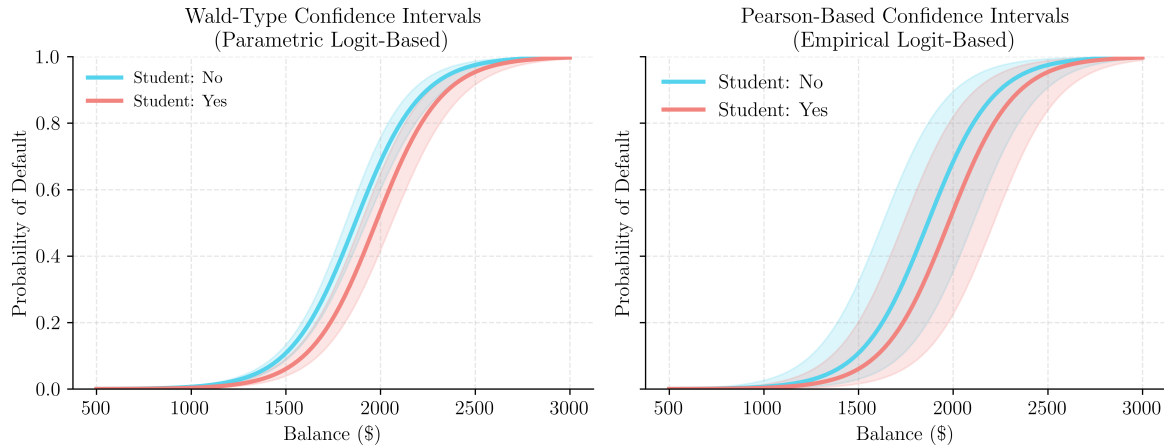


Figure 2.7: Comparison of uncertainty quantification methods for logistic regression on the `Default` dataset. Both models predict the probability of credit card default using *balance*, *income*, and *student status* as predictors. All intervals are constructed on the **logit (log-odds) scale** and then transformed back to probabilities for visualization. The left panel shows **Wald-type confidence intervals**, which reflect parametric uncertainty under the logistic model. The right panel shows **Pearson-based empirical intervals**, calibrated from residual variability on a held-out calibration set. While these logit-based empirical intervals appear smoother and more symmetric, they quantify uncertainty in the predicted *log-odds*, not in the binary outcomes themselves. On the probability scale, the corresponding Pearson-based intervals achieve approximately 95% empirical coverage but are wider and more asymmetric.

2.7 Later Developments: From Statistics to Machine Learning

By the 1970s, logistic regression had established itself as the standard approach for binary classification in statistics. What followed was a remarkable transformation: the migration of these ideas from statistical inference to computational learning, and ultimately to the neural networks that dominate modern machine learning.

The Computational Revolution (1970s-1980s)

The 1970s marked a turning point where logistic regression transitioned from a statistical technique requiring manual calculation to a computational tool. Two developments were particularly significant:

Generalized Linear Models (1972): Nelder and Wedderburn [19] introduced the GLM framework, unifying logistic regression with other models through the concept of link functions. This abstraction revealed that logistic regression was not an isolated technique but part of a broader mathematical structure. The GLM framework provided efficient iteratively reweighted least squares (IRLS) algorithms that made fitting logistic models computationally tractable.

Discrete Choice Theory: Daniel McFadden’s work on multinomial logistic regression and discrete choice analysis [18] extended logistic regression beyond binary outcomes. His theoretical contributions, which earned him the Nobel Prize in Economics (2000), showed how the logistic function naturally emerges from assumptions about random utility maximization. This connected statistical modeling to economic theory and opened new applications in transportation, marketing, and policy analysis.

The Machine Learning Era (1980s–2000s)

As computing power increased, logistic regression found new life in machine learning, where the focus shifted from inference to prediction:

Regularization Methods: The introduction of ridge regression and later the LASSO (Least Absolute Shrinkage and Selection Operator) in the 1990s addressed overfitting in high-dimensional settings. Regularized logistic regression became essential for handling datasets where the number of features approached or exceeded the number of observations—a common scenario in genomics, text classification, and early web applications.

Ensemble Methods: The 2000s saw logistic regression incorporated into ensemble frameworks. Boosting algorithms like AdaBoost and later gradient boosting machines combined multiple weak learners (often decision stumps) with logistic loss functions. This marriage of tree-based methods and logistic regression produced some of the most successful predictive models in practice.

Large-Scale Applications: The rise of internet companies in the late 1990s and 2000s created unprecedented demand for scalable classification algorithms. Logistic regression, with its convex loss function and efficient optimization, became the workhorse of spam filtering, ad click prediction, and recommendation systems. Companies like Google and Facebook deployed logistic regression models on billions of data points, driving innovations in distributed computing.

Contemporary Perspectives

Today, when you apply for a credit card and receive an instant decision, when your email filters spam with uncanny accuracy, or when a medical algorithm helps diagnose disease, you're witnessing the culmination of this remarkable journey. From Fisher's maximum likelihood framework to Berkson's logistic simplification, from Cox's mathematical refinements to McFadden's economic applications, these contributions have evolved into the algorithms that power modern decisioning.

The logistic function—an “underdog” once dismissed as inferior to its probit rival—now powers the decision engines of the modern economy. In the end, Berkson's vision proved prophetic: sometimes the “inferior” alternative becomes the foundation upon which entire multi-billion industries are built.

References

- [1] Joseph Berkson. ‘Application of the logistic function to bio-assay’. In: *Journal of the American Statistical Association* 39.227 (1944), pp. 357–365.
- [2] Chester I. Bliss. ‘The calculation of the dosage-mortality curve’. In: *Annals of Applied Biology* 22.1 (1935), pp. 134–167.
- [3] Chester I. Bliss. ‘The method of probits’. In: *Science* 79.2037 (1934), pp. 38–39.
- [4] Stephen R. Cole, Haitao Chu, and Sander Greenland. ‘Maximum likelihood, profile likelihood, and penalized likelihood: a primer’. In: *American Journal of Epidemiology* 179.2 (2014), pp. 252–260. DOI: [10.1093/aje/kwt245](https://doi.org/10.1093/aje/kwt245).
- [5] David Collett. *Modelling Binary Data*. 2nd. London: Chapman and Hall/CRC, 2003.
- [6] J. S. Cramer. ‘Sur la loi de la population’. In: *Annales de mathématiques pures et appliquées* (1825). Early work on logistic function for population growth.
- [7] David J. Finney. ‘A help to early users of probits’. In: *Annals of Applied Biology* 164.1 (2013), pp. 8–10.

- [8] David J. Finney. *Probit Analysis*. 1st. Cambridge, UK: Cambridge University Press, 1947.
- [9] R. A. Fisher. ‘On an Absolute Criterion for Fitting Frequency Curves’. In: *Statistical Science* 12.1 (1997). Reprint of 1912 paper, pp. 39–41.
- [10] Ronald A. Fisher. ‘Appendix to Bliss C. I. The case of zero survivors’. In: *Annals of Applied Biology* 22 (1935), pp. 164–165.
- [11] Ronald A. Fisher. ‘On the mathematical foundations of theoretical statistics’. In: *Philosophical Transactions of the Royal Society of London Series A* 222.594-604 (1922), pp. 309–368.
- [12] Andrew Gelman et al. *Bayesian Data Analysis*. 3rd. Boca Raton, FL: CRC Press, 2013.
- [13] Irving John Good. *Good thinking: the foundations of probability and its applications*. University of Minnesota Press, 1983.
- [14] David W. Hosmer and Stanley Lemeshow. *Applied Logistic Regression*. 2nd. New York: John Wiley & Sons, 2000.
- [15] Harold Jeffreys. *Theory of Probability*. 1st. Oxford: Oxford University Press, 1939.
- [16] Valery Manokhin. *Practical guide to applied conformal prediction in Python*. <https://github.com/valeman/awesome-conformal-prediction>. Comprehensive practical guide to implementing conformal prediction methods in Python. 2023.
- [17] Peter McCullagh and John A. Nelder. *Generalized Linear Models*. 2nd. Chapman & Hall, 1989.
- [18] Daniel McFadden. ‘Conditional logit analysis of qualitative choice behavior’. In: *Frontiers in Econometrics*. Ed. by P. Zarembka. Academic Press, 1973, pp. 105–142.
- [19] John A. Nelder and Robert W. M. Wedderburn. ‘Generalized linear models’. In: *Journal of the Royal Statistical Society Series A* 135.3 (1972), pp. 370–384.
- [20] Yudi Pawitan. *In All Likelihood: Statistical Modelling and Inference Using Likelihood*. Oxford: Oxford University Press, 2001.
- [21] David Sankoff and William Labov. ‘On the uses of variable rules’. In: *Language in Society* 8.2-3 (1979), pp. 189–222.
- [22] David J. Spiegelhalter and Robert P. Knill-Jones. ‘Statistical and knowledge-based approaches to clinical decision-support systems, with an application in gastroenterology’. In: *Journal of the Royal Statistical Society: Series A* 147.1 (1984), pp. 35–58.
- [23] Ryan J. Tibshirani. *Conformal Prediction*. Lecture notes, Advanced Topics in Statistical Learning, Spring 2023, Carnegie Mellon University. Available at: <https://www.stat.cmu.edu/~ryantibs/statml/>. 2023.
- [24] Dermot Turing. *Prof: Alan Turing Decoded*. Stroud, GBR: The History Press, 2016.